

EMBEDDED SYSTEM AND DESIGN

EXPERIMENT NO. 7

WATERING PLANT SYSTEM

Name: *Raphael S. Nano*

Course/Section: *CPE/60P / BI*

Date of Performance: *September 24-27, 2023*

Date of Submission: *September 27, 2023*

Ma'am Cyrel Manlises
Instructor

DISCUSSION

The quality of soil is very crucial in maintaining the quality of plants, and factors that improve soil quality include sunlight, minerals, and moisture. Depending on the plant, the moisture content of the soil may be varied. Some may need a lot of water, while others survive in little to no water. Hence, for this experiment, a system of watering plants is developed using Arduino UNO, a servo motor, a liquid crystal display, and a moisture sensor, wherein the moisture percentage of the soil will be considered as the determinant for allowing water to flow.



Figure 1. Final Out put for Experiment 7

The materials used for this experiment are the Arduino UNO, the SG90 servo motor, a QAPASS LiquidCrystal display, a YL-100 moisture sensor, and 4 plants, namely watermelon succulent, sasnsevieria, rosso and epicia. For the code, external libraries Servo.h and LiquidCrystal_I2C were utilized for the servo motor and the lcd, respectively.

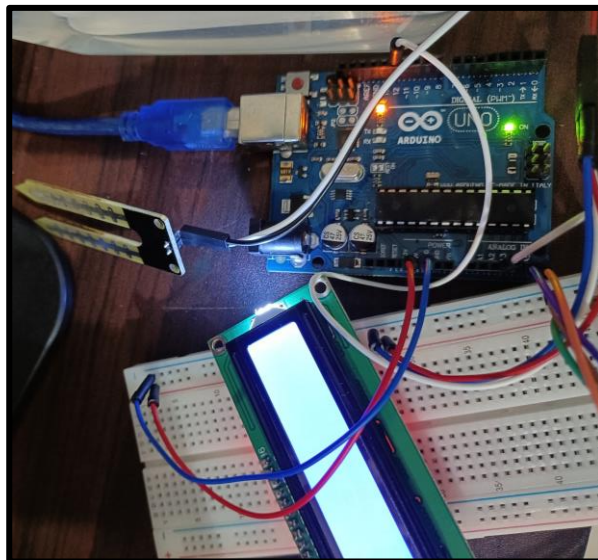


Figure 2. Main Components for the Experiment

The seventh experiment has a main goal of replicating a watering plant system, wherein the plants will be watered only when the detected soil moisture could be considered dry or below normal. So, the factors crucial for this experiment are the detection of the moisture

percentage of the soil and the movement of servo motor to open and close the replicated hose. For the YL-100 moisture sensor, the function `readMoist()` was created. Within the function, the analog pin was triggered, and the moisture percentage was computed by dividing the value to 1023, multiplying it by 100, and subtracting it to 100. The percentages needed for the plants were also recorded. According to Rutherford, Groves, and Sajeva (2018), although the 4 plants are different in appearances, most succulent plants do not need that much water. Hence little moisture is required for the experiment, and that little moisture could be attributed to Normal, which is tested to be 50 to 60 percent. The percentages for the ranges of soil characteristics were tested wherein water was proportionally added to the plants. As seen in Figures 4 to 7, for "Dry", the percentage was 20 to 40 percent. For "Below Normal," the percentage was 40 to 50 percent. "Normal" moisture content was 50 to 60 percent, while "Drowning" or

"Submerged" moisture content was 60 to 70 percent.

```
23 float readMoist() {  
24     float moist_percent;  
25     int sens_val;  
26     sens_val = analogRead(sensor);  
27     moist_percent = (100 - ((sens_val / 1023.00) * 100));  
28     Serial.println(moist_percent);  
29     return moist_percent;  
30 }
```

Figure 3. readMoist() function

```
42 void loop() {  
43     lcd.backlight();  
44     delay(5000);  
45     float readVal = readMoist();  
46     if ((readVal > 20) && (readVal < 40)) {  
47         lcd.setCursor(0, 0);  
48         lcd.print("Moisture Content: ");  
49         lcd.setCursor(5, 1);  
50         lcd.print(readVal);  
51         delay(5000);  
52         lcd.clear();  
53         lcd.setCursor(5, 1);  
54         lcd.print("Too Dry");  
55         openHose();  
56         delay(3000);  
57         closeHose();  
58         delay(100);  
}
```

Figure 4. Sample 1 - "Too Dry" Condition

```

59     } else if ((readVal > 40) && (readVal < 50)) {
60         lcd.setCursor(0, 0);
61         lcd.print("Moisture Content");
62         lcd.setCursor(5, 1);
63         lcd.print(readVal);
64         delay(2000);
65         lcd.setCursor(2, 1);
66         lcd.print("Below Normal");
67         openHose();
68         delay(3000);
69         closeHose();
70         delay(100);

```

Figure 5. Sample 2 - "Below Normal" Condition

```

71     } else if ((readVal > 50) && (readVal < 60)) {
72         lcd.setCursor(0, 0);
73         lcd.print("Moisture Content");
74         lcd.setCursor(5, 1);
75         lcd.print(readVal);
76         delay(5000);
77         lcd.setCursor(5, 1);
78         lcd.print("Normal");
79         delay(2000);

```

Figure 6. Sample 3 - "Normal" Condition

```

80     } else if ((readVal > 60) && (readVal < 70)) {
81         lcd.setCursor(0, 0);
82         lcd.print("Moisture Content");
83         lcd.setCursor(5, 1);
84         lcd.print(readVal);
85         delay(5000);
86         lcd.setCursor(4, 1);
87         lcd.print("Drowning!");
88         delay(5000);

```

Figure 7. Sample 4 - "Drowning" Condition

For the servo motor, the closeHose() and openHose() functions were employed, wherein the set initial position of the motor was 160 degrees. The closeHose() rotates the motor to 160 degrees, folding the straw upwards to stop water flow, and the openHose() rotates the motor to 100 degrees, unfolding the straw to allow water flow.

```
32 void closeHose() {  
33     servo.write(160);  
34     delay(10);  
35 }  
36  
37 void openHose() {  
38     servo.write(100);  
39     delay(10);  
40 }
```

Figure 8. Code for closeHose() and openHose() function

Lastly, for the liquid crystal display, the LiquidCrystal_I2C library made the output more convenient by simply printing the required output with the print() function. The lines in the output were also manipulated with rows and columns. The ranges and their corresponding percentages were evident in the output, displaying Dry, Below Normal, Normal, and Drowning in the LCD.

```

48     lcd.print("Moisture Content: ");
49     lcd.setCursor(5, 1);
50     lcd.print(readVal);
51     delay(5000);
52     lcd.clear();
53     lcd.setCursor(5, 1);
54     lcd.print("Too Dry");

```

Figure 9. Code for LCD

Hence, for the loop, it is the accumulation of all of the aforementioned functions, together with their respective conditions of percentages, out put display, and sensed value.

```

42 void loop() {
43     lcd.backlight();
44     delay(5000);
45     float readVal = readMoist();
46     if ((readVal > 20) && (readVal < 40)) {
47         lcd.setCursor(0, 0);
48         lcd.print("Moisture Content: ");
49         lcd.setCursor(5, 1);
50         lcd.print(readVal);
51         delay(5000);
52         lcd.clear();
53         lcd.setCursor(5, 1);
54         lcd.print("Too Dry");
55         openHose();
56         delay(3000);
57         closeHose();
58         delay(100);
59     } else if ((readVal > 40) && (readVal < 50)) {
60         lcd.setCursor(0, 0);
61         lcd.print("Moisture Content");
62         lcd.setCursor(5, 1);
63         lcd.print(readVal);
64         delay(2000);
65         lcd.setCursor(2, 1);
66         lcd.print("Below Normal");
67         openHose();
68         delay(3000);
69         closeHose();
70         delay(100);

```

Figure 10. Loop Code - Part 1

```

71     } else if ((readVal > 50) && (readVal < 60)) {
72         lcd.setCursor(0, 0);
73         lcd.print("Moisture Content");
74         lcd.setCursor(5, 1);
75         lcd.print(readVal);
76         delay(5000);
77         lcd.setCursor(5, 1);
78         lcd.print("Normal");
79         delay(2000);
80     } else if ((readVal > 60) && (readVal < 70)) {
81         lcd.setCursor(0, 0);
82         lcd.print("Moisture Content");
83         lcd.setCursor(5, 1);
84         lcd.print(readVal);
85         delay(5000);
86         lcd.setCursor(4, 1);
87         lcd.print("Drowning!");
88         delay(5000);
89     }
90     lcd.clear();
91 }

```

Figure 11. Loop Code - Part 2

For the final output of the experiment, the process of the automatic watering plant system was accurately demonstrated. For the first plant, which was the sansevieria, the change in the values of the moisture percentage was demonstrated, wherein from "Dry," it became "Normal" after adding water twice.

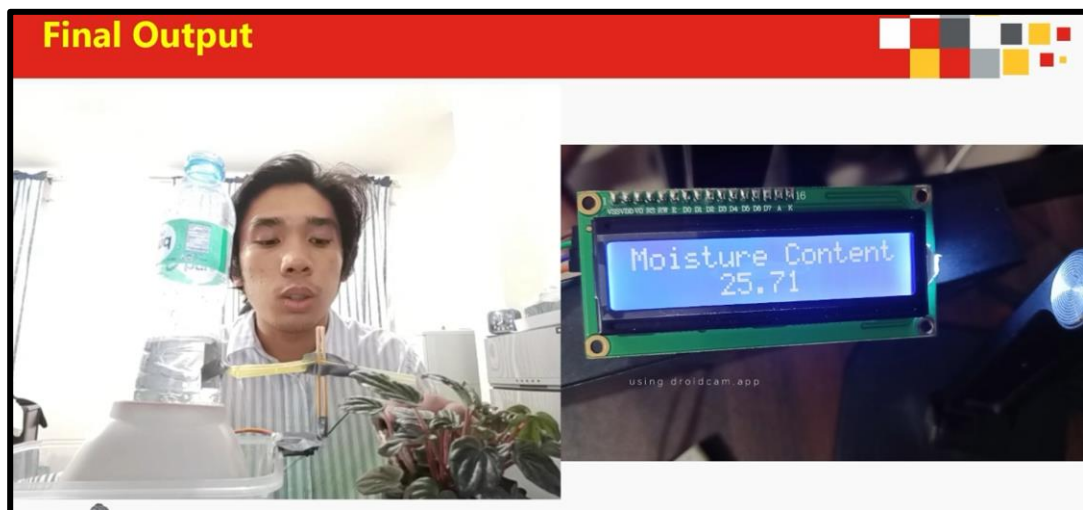


Figure 12. First Plant: Dry

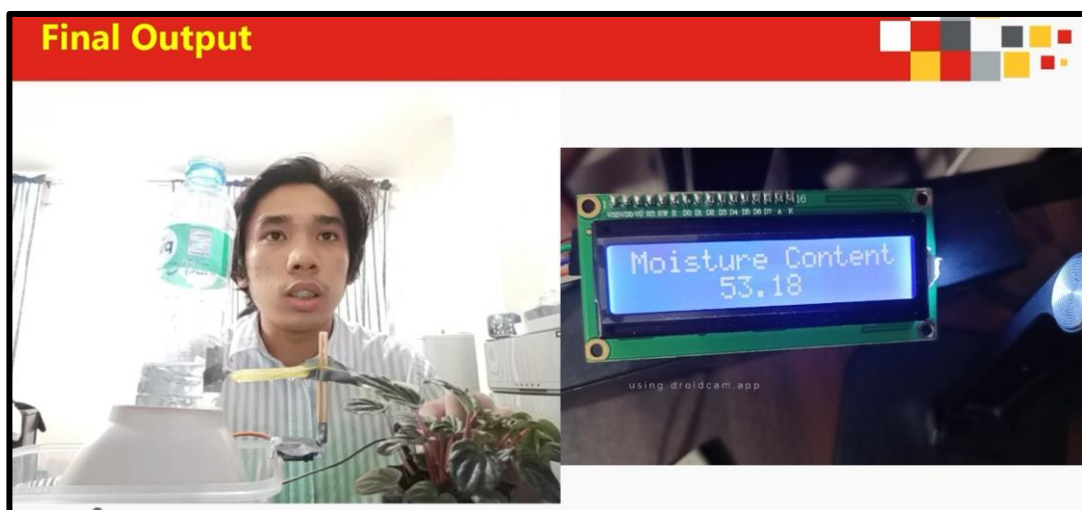


Figure 13. First Plant: Normal

Then, for the second plant, a change of status of the soil was evident, which showed a state of "Below Normal" to "Normal."



Figure 14. Second Plant: Below Normal

Finally, for the third and fourth plant, since they are normal and submerged, respectively, the motor did not move and the display in the LCD output the values respectively.

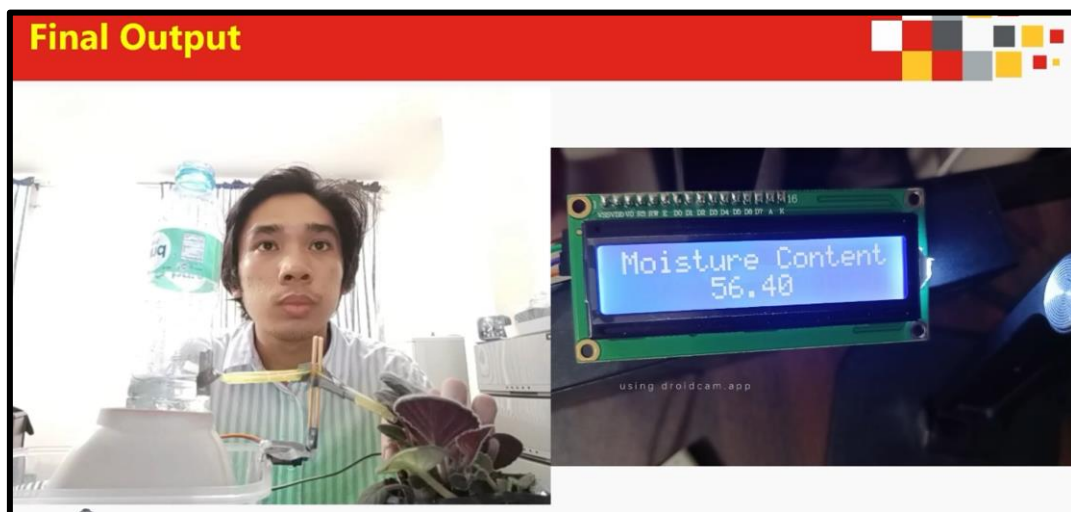


Figure 15. Third Plant: Normal

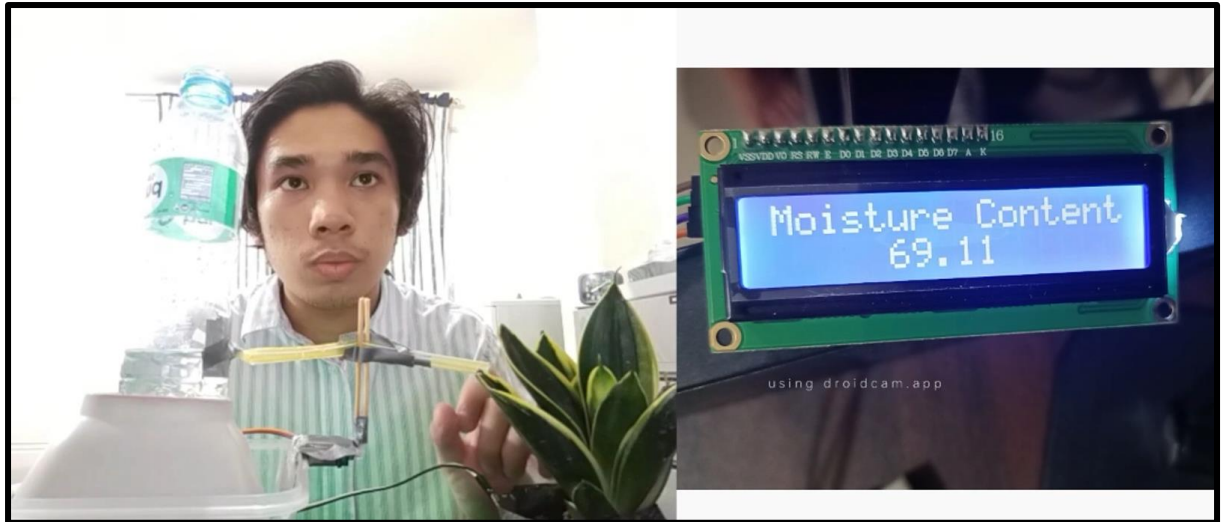


Figure 16. Fourth Plant: Submerged

CONCLUSION

In conclusion, considering the first, second, and third objectives, which concerns the overall procedure, the watering plant system works by detecting the moisture percentage of the soil using the moisture sensor, unfolding the straw to allow water flow if the soil moisture detected is Dry or Below Normal using the servo motor, and folding the straw if the soil moisture detected is Normal or Submerged or Drowning using the same servo motor. Through series connections, pin layouts, and sensor and motor triggering, the usage of microprocessors, logic circuits, and switching theory was illustrated. The watering plant system was successfully developed and exhibited in a basic handshaking as an outcome

of employing many variables, modules, and calling such functions to construct an embedded system.

REFERENCES

Rutherford, C., Groves, M., & Sajeve, M. (2018). Succulent Plants: A Guide to CITES-listed species. Rutherford Groves Publishing. Retrieved on September 27, 2023 from <https://core.ac.uk/download/pdf/141697558.pdf>